

Dokumentacja kontraktów API i funkcji backendowych

1. Informacje ogólne

1.1. Cel dokumentu

Dokument opisuje kontrakty komunikacji pomiędzy aplikacją mobilną Live GeoGuessr a backendem opartym na Firebase Cloud Functions.

Dokumentacja obejmuje:

- publiczne funkcje Firebase Cloud Functions,
- dane wejściowe i wyjściowe funkcji,
- wymagania dotyczące uwierzytelnienia,
- walidację danych,
- możliwe błędy,
- operacje wykonywane w Cloud Firestore,
- operacje wykonywane w Firebase Storage,
- operacje wykonywane w Firebase Authentication,
- wewnętrzne funkcje pomocnicze backendu.

Dokumentacja metod repozytoriów aplikacji zostanie dodana w kolejnych sekcjach.

2. Technologie

Element	Technologia
Aplikacja mobilna	Android, Kotlin
Backend	Firebase Cloud Functions 2nd Gen
Język backendu	TypeScript
Uwierzytelnianie	Firebase Authentication
Baza danych	Cloud Firestore
Pliki	Firebase Storage
Komunikacja	Firebase Callable Functions oraz HTTP
Region funkcji	us-central1
Maksymalna liczba instancji	10

Globalna konfiguracja funkcji:

```
setGlobalOptions({  
  region: "us-central1",
```

```
    maxInstances: 10,  
  });
```

3. Konwencje API

3.1. Typy funkcji

Backend udostępnia dwa typy funkcji:

Typ	Mechanizm	Zastosowanie
Callable Function	onCall	Komunikacja aplikacji mobilnej z backendem
HTTP Function	onRequest	Kontrola dostępności backendu

3.2. Uwierzytelnienie

Funkcje callable wymagające zalogowanego użytkownika korzystają z:

```
request.auth  
request.auth.uid
```

Identyfikator użytkownika pobierany jest przez funkcję pomocniczą:

```
requireAuthenticatedUid(request)
```

Brak uwierzytelnienia powoduje zwrócenie błędu:

```
unauthenticated
```

z komunikatem:

```
User must be logged in.
```

3.3. Format danych wejściowych

Dane wejściowe funkcji callable przekazywane są przez:

```
request.data
```

Przykład:

```
{  
  "postId": "examplePostId"  
}
```

3.4. Format odpowiedzi

Poprawna odpowiedź funkcji zwracana jest jako obiekt JSON.

Przykład:

```
{  
  "success": true  
}
```

3.5. Format błędów

Kontrolowane błędy backendowe zwracane są za pomocą klasy:

HttpsError

Przykład:

```
throw new HttpsError(  
  "invalid-argument",  
  "Invalid latitude."  
);
```

4. Kody błędów

Kod Firebase	Znaczenie
unauthenticated	Użytkownik nie jest zalogowany
permission-denied	Użytkownik nie ma uprawnień do wykonania operacji
invalid-argument	Przekazano nieprawidłowe lub niekompletne dane
not-found	Nie znaleziono wskazanego zasobu
already-exists	Zasób lub relacja już istnieje
failed-precondition	Nie zostały spełnione warunki wykonania operacji
internal	Wystąpił nieoczekiwany błąd serwera

5. Funkcje backendowe

5.1. health

Opis

Funkcja służy do sprawdzenia, czy backend Firebase Cloud Functions jest dostępny i poprawnie odpowiada na żądania HTTP.

Typ funkcji

Firebase HTTP Function

Implementacja:

onRequest

Uwierzytelnienie

Uwierzytelnienie nie jest wymagane.

Dane wejściowe

Funkcja nie wymaga żadnych danych wejściowych.

Metoda HTTP nie jest ograniczona w kodzie funkcji.

Przebieg działania

1. Funkcja odbiera żądanie HTTP.
2. Zapisuje w logach informację Health check called.
3. Zwraca informacje o stanie backendu.

Operacje Firestore

Brak operacji na Cloud Firestore.

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

Pole	Typ	Opis
status	string	Stan działania backendu
app	string	Nazwa aplikacji
backend	string	Nazwa wykorzystywanej technologii backendowej

Przykład:

```
{  
  "status": "ok",  
  "app": "Live GeoGuessr",  
  "backend": "Firebase Cloud Functions"  
}
```

5.2. sendFriendRequest

Opis

Funkcja wysyła zaproszenie do znajomych do wskazanego użytkownika.

Identyfikator zaproszenia jest generowany na podstawie identyfikatorów obu użytkowników. Identyfikatory są sortowane, dzięki czemu dla tej samej pary użytkowników zawsze powstaje ten sam identyfikator dokumentu.

Przykład:

```
uidA_uidB
```

Typ funkcji

Firestore Callable Function

Uwierzytelnienie

Wymagane	Źródło identyfikatora użytkownika
----------	-----------------------------------

Tak	request.auth.uid
-----	------------------

Dane wejściowe

Pole	Typ	Wymagane	Opis
toUid	string	Tak	UID użytkownika, do którego wysyłane jest zaproszenie

Przykład:

```
{
  "toUid": "recipientUserId"
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy toUid zostało przekazane.
3. Czy toUid nie jest pustym ciągiem znaków.
4. Czy użytkownik nie próbuje dodać samego siebie.
5. Czy profil nadawcy istnieje.
6. Czy profil odbiorcy istnieje.
7. Czy oczekujące zaproszenie już istnieje.
8. Czy użytkownicy nie są już znajomymi.

9. Czy relacja znajomości nie istnieje w podkolekcji friends.

Przebieg działania

1. Pobranie UID zalogowanego użytkownika jako fromUid.
2. Pobranie i walidacja toUid.
3. Wygenerowanie identyfikatora zaproszenia.
4. Pobranie publicznych danych obu użytkowników.
5. Rozpoczęcie transakcji Firestore.
6. Sprawdzenie istniejącego zaproszenia.
7. Sprawdzenie istniejącej relacji znajomości.
8. Utworzenie dokumentu zaproszenia ze statusem pending.
9. Zwrócenie identyfikatora zaproszenia.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	users/{fromUid}	Pobranie danych nadawcy
Odczyt	users/{toUid}	Pobranie danych odbiorcy
Odczyt	friendRequests/{requestId}	Sprawdzenie istniejącego zaproszenia
Odczyt	users/{fromUid}/friends/{toUid}	Sprawdzenie istniejącej znajomości
Zapis	friendRequests/{requestId}	Utworzenie zaproszenia

Zapisywane dane zaproszenia

```
{
  "fromUid": "senderUid",
  "toUid": "recipientUid",
  "fromNickname": "SenderNickname",
  "fromDisplayName": "Sender Display Name",
  "fromPhotoUrl": null,
  "toNickname": "RecipientNickname",
  "toDisplayName": "Recipient Display Name",
  "toPhotoUrl": null,
  "status": "pending",
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

Pole	Typ	Opis
success	boolean	Informacja o powodzeniu operacji
requestId	string	Identyfikator utworzonego zaproszenia

Przykład:

```
{
  "success": true,
  "requestId": "uidA_uidB"
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano toUid	Missing toUid.
invalid-argument	Użytkownik próbuje dodać siebie	Cannot add yourself.
not-found	Profil jednego z użytkowników nie istnieje	User not found.
already-exists	Oczekujące zaproszenie już istnieje	Friend request already exists.
already-exists	Użytkownicy są już znajomymi	Users are already friends.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.3. acceptFriendRequest

Opis

Funkcja akceptuje oczekujące zaproszenie do znajomych.

Po zaakceptowaniu zaproszenia tworzona jest dwukierunkowa relacja znajomości w podkolekcjach obu użytkowników.

Typ funkcji

Firebase Callable Function

Uwierzytelnienie

Wymagane Źródło identyfikatora użytkownika

Tak request.auth.uid

Dane wejściowe

Pole	Typ	Wymagane	Opis
requestId	string	Tak	Identyfikator zaproszenia do znajomych

Przykład:

```
{  
  "requestId": "uidA_uidB"  
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy requestId zostało przekazane.
3. Czy dokument zaproszenia istnieje.
4. Czy dane dokumentu zaproszenia są poprawne.
5. Czy zalogowany użytkownik jest odbiorcą zaproszenia.
6. Czy status zaproszenia wynosi pending.

Przebieg działania

1. Pobranie UID zalogowanego użytkownika.
2. Pobranie i walidacja requestId.
3. Rozpoczęcie transakcji Firestore.
4. Pobranie dokumentu zaproszenia.
5. Weryfikacja odbiorcy zaproszenia.
6. Weryfikacja statusu zaproszenia.
7. Utworzenie relacji znajomości po stronie nadawcy.
8. Utworzenie relacji znajomości po stronie odbiorcy.
9. Zmiana statusu zaproszenia na accepted.
10. Zwiększenie liczników znajomych obu użytkowników, jeżeli relacja wcześniej nie istniała.
11. Zwrócenie informacji o powodzeniu operacji.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	friendRequests/{requestId}	Pobranie zaproszenia
Odczyt	users/{fromUid}/friends/{toUid}	Sprawdzenie istniejącej relacji
Zapis	users/{fromUid}/friends/{toUid}	Utworzenie relacji po stronie nadawcy
Zapis	users/{toUid}/friends/{fromUid}	Utworzenie relacji po stronie odbiorcy
Aktualizacja	friendRequests/{requestId}	Ustawienie statusu accepted
Aktualizacja	users/{fromUid}	Zwiększenie stats.friendsCount
Aktualizacja	users/{toUid}	Zwiększenie stats.friendsCount

Dokument relacji znajomości

```
{
  "uid": "friendUid",
  "nickname": "FriendNickname",
  "displayName": "Friend Display Name",
  "photoUrl": null,
  "addedAt": "serverTimestamp"
}
```

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

```
{
  "success": true
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano requestId	Missing requestId.
not-found	Zaproszenie nie istnieje	Friend request not found.
internal	Dokument zaproszenia nie	Invalid friend request.

Kod	Warunek	Komunikat
permission-denied	zawiera poprawnych danych Operację wykonuje użytkownik inny niż odbiorca	Only receiver can accept this request.
failed-precondition	Zaproszenie nie ma statusu pending	Request is not pending.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.4. rejectFriendRequest

Opis

Funkcja odrzuca oczekujące zaproszenie do znajomych.

Funkcję może wykonać wyłącznie odbiorca zaproszenia.

Typ funkcji

Firestore Callable Function

Uwierzytelnienie

Wymagane Źródło identyfikatora użytkownika

Tak request.auth.uid

Dane wejściowe

Pole	Typ	Wymagane	Opis
requestId	string	Tak	Identyfikator zaproszenia do znajomych

Przykład:

```
{
  "requestId": "uidA_uidB"
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy requestId zostało przekazane.

3. Czy dokument zaproszenia istnieje.
4. Czy dokument zawiera poprawne dane.
5. Czy zalogowany użytkownik jest odbiorcą zaproszenia.
6. Czy status zaproszenia wynosi pending.

Przebieg działania

1. Pobranie UID zalogowanego użytkownika.
2. Pobranie i walidacja requestId.
3. Rozpoczęcie transakcji Firestore.
4. Pobranie dokumentu zaproszenia.
5. Weryfikacja odbiorcy i statusu zaproszenia.
6. Ustawienie statusu rejected.
7. Aktualizacja pola updatedAt.
8. Zwrócenie informacji o powodzeniu operacji.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	friendRequests/{requestId}	Pobranie zaproszenia
Aktualizacja	friendRequests/{requestId}	Ustawienie statusu rejected

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

```
{  
  "success": true  
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano requestId	Missing requestId.
not-found	Zaproszenie nie istnieje	Friend request not found.
internal	Dokument zaproszenia nie zawiera poprawnych danych	Invalid friend request.
permission-denied	Operację wykonuje użytkownik inny niż odbiorca	Only receiver can reject this request.

Kod	Warunek	Komunikat
failed-precondition	Zaproszenie nie ma statusu pending	Request is not pending.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.5. removeFriend

Opis

Funkcja usuwa istniejącą relację znajomości pomiędzy zalogowanym użytkownikiem a wskazanym znajomym.

Relacja zostaje usunięta z podkolekcji friends obu użytkowników.

Typ funkcji

Firebase Callable Function

Uwierzytelnienie

Wymagane Źródło identyfikatora użytkownika

Tak request.auth.uid

Dane wejściowe

Pole	Typ	Wymagane	Opis
friendUid	string	Tak	UID usuwanego znajomego

Przykład:

```
{
  "friendUid": "friendUserId"
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy friendUid zostało przekazane.
3. Czy friendUid różni się od UID zalogowanego użytkownika.
4. Czy relacja znajomości istnieje po stronie zalogowanego użytkownika.

Przebieg działania

1. Pobranie UID zalogowanego użytkownika.
2. Pobranie i walidacja friendUid.
3. Wyznaczenie ścieżek obu dokumentów relacji.
4. Wyznaczenie identyfikatora dokumentu zaproszenia.
5. Rozpoczęcie transakcji Firestore.
6. Sprawdzenie istnienia relacji znajomości.
7. Usunięcie relacji z podkolekcji obu użytkowników.
8. Ustawienie statusu zaproszenia na removed.
9. Zmniejszenie liczników znajomych obu użytkowników.
10. Zwrócenie informacji o powodzeniu operacji.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	users/{uid}/friends/{friendUid}	Sprawdzenie relacji
Usunięcie	users/{uid}/friends/{friendUid}	Usunięcie relacji użytkownika
Usunięcie	users/{friendUid}/friends/{uid}	Usunięcie relacji drugiego użytkownika
Zapis z merge	friendRequests/{requestId}	Ustawienie statusu removed
Aktualizacja	users/{uid}	Zmniejszenie stats.friendsCount
Aktualizacja	users/{friendUid}	Zmniejszenie stats.friendsCount

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

```
{  
  "success": true  
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano friendUid	Missing friendUid.

Kod	Warunek	Komunikat
invalid-argument	UID znajomego jest równy UID użytkownika	Invalid friendUid.
not-found	Relacja znajomości nie istnieje	Friend relation not found.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.6. getGuessMapPreview

Opis

Funkcja przygotowuje początkowy widok mapy używany przed wykonaniem zgadywania lokalizacji posta.

Funkcja nie zwraca rzeczywistych współrzędnych posta. Zwracany środek mapy jest losowo przesunięty względem prawdziwej lokalizacji.

Typ funkcji

Firestore Callable Function

Uwierzytelnienie

Wymagane Źródło identyfikatora użytkownika

Tak	request.auth.uid
-----	------------------

Dane wejściowe

Pole	Typ	Wymagane	Opis
postId	string	Tak	Identyfikator posta
gameMode	string	Nie	Tryb gry; domyślnie CITY

Przykład:

```
{
  "postId": "examplePostId",
  "gameMode": "CITY"
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.

2. Czy postId zostało przekazane.
3. Czy wybrany tryb gry jest obsługiwany.
4. Czy dokument posta istnieje.
5. Czy współrzędne posta są poprawne.
6. Czy tryb gry przekazany przez aplikację zgadza się z trybem zapisanym w poście.

Przebieg działania

1. Weryfikacja uwierzytelnienia.
2. Pobranie i walidacja postId.
3. Pobranie konfiguracji trybu gry.
4. Pobranie dokumentu posta.
5. Walidacja rzeczywistych współrzędnych.
6. Sprawdzenie zgodności trybu gry.
7. Wygenerowanie przybliżonego środka mapy.
8. Zwrócenie parametrów początkowego widoku mapy.

Konfiguracja trybu CITY

Parametr	Wartość
Maksymalna odległość punktowana	2000 m
Maksymalna liczba punktów	10000
Początkowa średnica mapy	36000 m
Maksymalne przesunięcie środka mapy	30000 m
Wersja systemu punktacji	1

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	posts/{postId}	Pobranie lokalizacji i trybu gry posta

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

Pole	Typ	Opis
postId	string	Identyfikator posta
gameMode	string	Tryb gry
initialMapCenterLatitude	number	Szerokość geograficzna przybliżonego środka mapy
initialMapCenterLongitude	number	Długość geograficzna przybliżonego środka mapy

Pole	Typ	Opis
initialMapDiameterMeters	number	Początkowa średnica widoku mapy w metrach

Przykład:

```
{
  "postId": "examplePostId",
  "gameMode": "CITY",
  "initialMapCenterLatitude": 50.0412,
  "initialMapCenterLongitude": 21.9984,
  "initialMapDiameterMeters": 36000
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano postId	Missing postId.
invalid-argument	Nieobstugiwany tryb gry	Invalid game mode.
invalid-argument	Niepoprawna szerokość geograficzna	Invalid latitude.
invalid-argument	Niepoprawna długość geograficzna	Invalid longitude.
not-found	Post nie istnieje	Post not found.
failed-precondition	Tryb gry nie zgadza się z trybem posta	Game mode does not match post game mode.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.7. submitGuess

Opis

Funkcja zapisuje odpowiedź użytkownika dotyczącą lokalizacji wskazanego posta.

Funkcja oblicza odległość pomiędzy wskazaną lokalizacją a rzeczywistą lokalizacją posta, przyznaje punkty i aktualizuje statystyki użytkownika.

Jeden użytkownik może zgadnąć dany post tylko jeden raz.

Typ funkcji

Firestore Callable Function

Uwierzytelnienie

Wymagane Źródło identyfikatora użytkownika

Tak request.auth.uid

Dane wejściowe

Pole	Typ	Wymagane	Opis
postId	string	Tak	Identyfikator zgadywanego posta
guessedLatitude	number	Tak	Wybrana szerokość geograficzna
guessedLongitude	number	Tak	Wybrana długość geograficzna
gameMode	string	Nie	Tryb gry; domyślnie CITY

Przykład:

```
{
  "postId": "examplePostId",
  "guessedLatitude": 50.0351,
  "guessedLongitude": 21.9823,
  "gameMode": "CITY"
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy postId zostało przekazane.
3. Czy współrzędne wybrane przez użytkownika są poprawne.
4. Czy tryb gry jest obsługiwany.
5. Czy post istnieje.
6. Czy profil użytkownika istnieje.
7. Czy użytkownik nie zgadywał wcześniej tego posta.
8. Czy użytkownik nie jest właścicielem posta.
9. Czy rzeczywiste współrzędne posta są poprawne.
10. Czy tryb gry zgadza się z trybem zapisanym w poście.

Dozwolone zakresy współrzędnych

Wartość	Minimalna	Maksymalna
Szerokość geograficzna	-90	90
Długość geograficzna	-180	180

Przebieg działania

1. Pobranie UID zalogowanego użytkownika.
2. Pobranie danych wejściowych.
3. Walidacja współrzędnych i trybu gry.
4. Przygotowanie referencji dokumentów posta, użytkownika i odpowiedzi.
5. Rozpoczęcie transakcji Firestore.
6. Pobranie posta, profilu użytkownika i istniejącej odpowiedzi.
7. Sprawdzenie, czy użytkownik może zgadywać dany post.
8. Pobranie rzeczywistych współrzędnych posta.
9. Obliczenie odległości.
10. Obliczenie liczby punktów.
11. Utworzenie dokumentu odpowiedzi.
12. Aktualizacja statystyk użytkownika.
13. Zwrócenie wyniku.

Identyfikator odpowiedzi

Dokument odpowiedzi otrzymuje identyfikator:

`{uid}_{postId}`

Przykład:

`userId_examplePostId`

Obliczanie odległości

Odległość obliczana jest w metrach przy użyciu wzoru Haversine'a.

Przyjęty promień Ziemi:

`6371000 m`

Obliczanie punktów

Dla trybu CITY:

- maksymalna liczba punktów wynosi 10000,
- maksymalna punktowana odległość wynosi 2000 m,
- odległość równa 0 m daje 10000 punktów,
- odległość większa lub równa 2000 m daje 0 punktów,
- wyniki pośrednie są obliczane na podstawie współczynnika odległości,
- wynik jest zaokrąglany do liczby całkowitej.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	posts/{postId}	Pobranie posta
Odczyt	users/{uid}	Pobranie profilu i statystyk
Odczyt	guesses/{uid}_{postId}	Sprawdzenie wcześniejszej odpowiedzi
Zapis	guesses/{uid}_{postId}	Zapis wyniku zgadywania
Aktualizacja	users/{uid}	Aktualizacja statystyk użytkownika

Dokument odpowiedzi

```
{
  "id": "userId_postId",
  "userId": "userId",
  "postId": "postId",
  "gameMode": "CITY",
  "guessedLatitude": 50.0351,
  "guessedLongitude": 21.9823,
  "realLatitude": 50.0412,
  "realLongitude": 21.9984,
  "distanceMeters": 1250.5,
  "points": 4300,
  "maxPoints": 10000,
  "maxScoringDistanceMeters": 2000,
  "scoringVersion": 1,
  "createdAt": "serverTimestamp"
}
```

Aktualizowane statystyki użytkownika

Pole	Operacja
stats.pointsTotal	Zwiększenie o liczbę zdobytych punktów
stats.guessesCount	Zwiększenie o 1
stats.bestGuessMeters	Ustawienie nowego najlepszego wyniku, jeżeli odległość jest mniejsza
updatedAt	Ustawienie czasu serwera

Operacje Firebase Storage

Brak operacji na Firebase Storage.

Odpowiedź poprawna

Pole	Typ	Opis
guessId	string	Identyfikator odpowiedzi

Pole	Typ	Opis
postId	string	Identyfikator posta
gameMode	string	Tryb gry
distanceMeters	number	Odległość od prawdziwej lokalizacji
points	number	Liczba zdobytych punktów
maxPoints	number	Maksymalna liczba punktów
maxScoringDistanceMeters	number	Maksymalna punktowana odległość
scoringVersion	number	Wersja systemu punktacji
realLatitude	number	Rzeczywista szerokość geograficzna
realLongitude	number	Rzeczywista długość geograficzna
guessedLatitude	number	Wybrana szerokość geograficzna
guessedLongitude	number	Wybrana długość geograficzna

Przykład:

```
{
  "guessId": "userId_examplePostId",
  "postId": "examplePostId",
  "gameMode": "CITY",
  "distanceMeters": 1250.5,
  "points": 4300,
  "maxPoints": 10000,
  "maxScoringDistanceMeters": 2000,
  "scoringVersion": 1,
  "realLatitude": 50.0412,
  "realLongitude": 21.9984,
  "guessedLatitude": 50.0351,
  "guessedLongitude": 21.9823
}
```

Możliwe błędy

Kod	Warunek	Komunikat
unauthenticated	Użytkownik nie jest zalogowany	User must be logged in.
invalid-argument	Nie przekazano postId	Missing postId.
invalid-argument	Niepoprawna szerokość	Invalid latitude.

Kod	Warunek	Komunikat
	geograficzna	
invalid-argument	Niepoprawna długość geograficzna	Invalid longitude.
invalid-argument	Nieobsługiwany tryb gry	Invalid game mode.
not-found	Post nie istnieje	Post not found.
failed-precondition	Profil użytkownika nie istnieje	User profile not found.
already-exists	Użytkownik wcześniej zgadywał ten post	User has already guessed this post.
permission-denied	Użytkownik próbuje zgadnąć własny post	Cannot guess your own post.
failed-precondition	Tryb gry nie zgadza się z trybem posta	Game mode does not match post game mode.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

5.8. deleteAccount

Opis

Funkcja trwale usuwa konto zalogowanego użytkownika oraz wszystkie powiązane z nim dane.

Usuwane są:

- profil użytkownika,
- posty użytkownika,
- odpowiedzi użytkownika,
- odpowiedzi innych użytkowników dotyczące usuwanych postów,
- zaproszenia do znajomych wysłane przez użytkownika,
- zaproszenia do znajomych otrzymane przez użytkownika,
- relacje znajomości,
- zdjęcie profilowe,
- zdjęcia postów,
- konto Firebase Authentication.

Typ funkcji

Firebase Callable Function

Konfiguracja funkcji

Parametr	Wartość
Limit czasu	540 sekund
Pamięć	512 MiB
Region	us-central1

Uwierzytelnienie

Wymagane	Źródło identyfikatora użytkownika
Tak	<code>request.auth.uid</code>

Dane wejściowe

Pole	Typ	Wymagane	Opis
<code>confirm</code>	<code>boolean</code>	Tak	Musi mieć wartość <code>true</code>

Przykład:

```
{  
  "confirm": true  
}
```

Walidacja danych

Funkcja sprawdza:

1. Czy użytkownik jest zalogowany.
2. Czy pole `confirm` ma dokładnie wartość logiczną `true`.

Wartości tekstowe, takie jak `"true"`, nie są akceptowane.

Przebieg działania

1. Pobranie UID zalogowanego użytkownika.
2. Sprawdzenie potwierdzenia usunięcia konta.
3. Pobranie postów użytkownika.
4. Pobranie odpowiedzi użytkownika.
5. Pobranie wysłanych zaproszeń.
6. Pobranie otrzymanych zaproszeń.
7. Pobranie listy znajomych.
8. Pobranie wszystkich odpowiedzi dotyczących postów użytkownika.
9. Określenie użytkowników, których statystyki wymagają aktualizacji.
10. Usunięcie plików z Firebase Storage.
11. Przygotowanie listy dokumentów Firestore do usunięcia.
12. Usunięcie dokumentów przy użyciu `BulkWriter`.

13. Ponowne obliczenie liczby znajomych byłych znajomych użytkownika.
14. Ponowne obliczenie statystyk zgadywania użytkowników, których odpowiedzi zostały usunięte.
15. Usunięcie konta z Firebase Authentication.
16. Zapisanie informacji o operacji w logach.
17. Zwrócenie informacji o powodzeniu operacji.

Operacje Firestore

Operacja	Ścieżka	Opis
Odczyt	posts, filtr <code>userId == uid</code>	Pobranie postów użytkownika
Odczyt	guesses, filtr <code>userId == uid</code>	Pobranie odpowiedzi użytkownika
Odczyt	guesses, filtr <code>postId == postId</code>	Pobranie odpowiedzi dotyczących usuwanych postów
Odczyt	friendRequests, filtr <code>fromUid == uid</code>	Pobranie wysłanych zaproszeń
Odczyt	friendRequests, filtr <code>toUid == uid</code>	Pobranie otrzymanych zaproszeń
Odczyt	users/{uid}/friends	Pobranie relacji znajomości
Usunięcie	posts/{postId}	Usunięcie postów
Usunięcie	guesses/{guessId}	Usunięcie odpowiedzi
Usunięcie	friendRequests/{requestId}	Usunięcie zaproszeń
Usunięcie	users/{uid}/friends/{friendUid}	Usunięcie własnych relacji
Usunięcie	users/{friendUid}/friends/{uid}	Usunięcie relacji po stronie znajomych
Usunięcie	users/{uid}	Usunięcie profilu
Aktualizacja	users/{friendUid}	Ponowne obliczenie <code>stats.friendsCount</code>
Aktualizacja	users/{guessUserUid}	Ponowne obliczenie statystyk zgadywania

Ponownie obliczane statystyki znajomych

`stats.friendsCount`

Wartość jest ustawiana na rzeczywistą liczbę dokumentów w podkolekcji:

`users/{uid}/friends`

Ponownie obliczane statystyki zgadywania

Pole	Sposób obliczenia
<code>stats.guessesCount</code>	Liczba pozostałych dokumentów użytkownika w <code>guesses</code>
<code>stats.pointsTotal</code>	Suma punktów z pozostałych odpowiedzi
<code>stats.bestGuessMeters</code>	Najmniejsza odległość z pozostałych odpowiedzi

Operacje Firebase Storage

Operacja	Ścieżka	Opis
Usunięcie	<code>avatars/{uid}/</code>	Usunięcie wszystkich plików profilu
Usunięcie	<code>posts/{uid}/</code>	Usunięcie wszystkich zdjęć postów

Pliki są usuwane partiami po maksymalnie 50 elementów.

Operacje Firebase Authentication

Operacja	Opis
<code>deleteUser(uid)</code>	Trwałe usunięcie konta użytkownika

Odpowiedź poprawna

```
{  
  "success": true  
}
```

Możliwe błędy

Kod	Warunek	Komunikat
<code>unauthenticated</code>	Użytkownik nie jest zalogowany	User must be logged in.
<code>invalid-argument</code>	Usunięcie konta nie zostało potwierdzone	Account deletion was not confirmed.
<code>internal</code>	Wystąpił nieobsłużony błąd podczas usuwania	ACCOUNT_DELETION_FAILED

Uwagi techniczne

Usuwanie konta składa się z kilku kolejnych operacji na Firestore, Storage i Authentication. Cały proces nie jest wykonywany jako jedna transakcja obejmująca wszystkie usługi Firebase.

Powiązana metoda repozytorium

Do uzupełnienia po dodaniu dokumentacji repozytoriów.

6. Funkcje pomocnicze backendu

6.1. Funkcje autoryzacji i walidacji

Funkcja	Opis	Możliwy błąd
<code>requireAuthenticatedUid</code>	Pobiera UID zalogowanego użytkownika	<code>unauthenticated</code>
<code>requireString</code>	Konwertuje wartość na tekst, usuwa białe znaki i sprawdza, czy wynik nie jest pusty	<code>invalid-argument</code>
<code>assertValidCoordinates</code>	Sprawdza zakres szerokości i długości geograficznej	<code>invalid-argument</code>

6.2. Funkcje Firestore

Funkcja	Opis
<code>getFriendRequestId</code>	Generuje deterministyczny identyfikator zaproszenia na podstawie dwóch UID
<code>getUserPublicData</code>	Pobiera publiczne dane użytkownika
<code>deleteDocumentReferences</code>	Usuwa unikalne dokumenty Firestore za pomocą BulkWriter

Publiczne dane użytkownika

```
type UserPublicData = {  
  uid: string;  
  nickname: string;  
  displayName: string;  
  photoUrl: string | null;  
};
```

Jeżeli użytkownik nie posiada wartości `nickname` lub `displayName`, stosowana jest wartość zastępcza:

Player

6.3. Funkcje systemu punktacji

Funkcja	Opis
<code>getGameModeConfig</code>	Zwraca konfigurację trybu gry
<code>calculateDistanceMeters</code>	Oblicza odległość pomiędzy współrzędnymi
<code>calculatePoints</code>	Oblicza liczbę punktów na podstawie

Funkcja	Opis
<code>createApproximateMapCenter</code>	odległości Generuje losowo przesunięty środek początkowej mapy

Obsługiwane tryby gry

`type` `GameModeType` = `"CITY"`;

Backend obsługuje obecnie wyłącznie tryb:

CITY

Konfiguracja trybu CITY

```
{
  "type": "CITY",
  "displayName": "Miasto",
  "maxScoringDistanceMeters": 2000,
  "maxPoints": 10000,
  "initialMapDiameterMeters": 36000,
  "initialMapOffsetMaxMeters": 30000,
  "scoringVersion": 1
}
```

7. Repozytoria aplikacji

7.1. AuthRepository

Odpowiedzialność

Repozytorium odpowiada za uwierzytelnianie użytkowników, rejestrację, logowanie, ponowne uwierzytelnienie, określenie dostawcy logowania, wylogowanie oraz usunięcie konta.

Zależności

Zależność	Zastosowanie
<code>FirebaseAuth</code>	Logowanie, rejestracja, ponowne uwierzytelnienie i wylogowanie
<code>FirebaseFunctions</code>	Wywołanie funkcji <code>deleteAccount</code>
<code>CredentialManager</code>	Pobranie danych logowania Google
<code>GoogleIdTokenCredential</code>	Odczyt tokenu Google ID

Zależność	Zastosowanie
UserRepository	Utworzenie dokumentu użytkownika po rejestracji
FirebaseApp	Pobranie instancji usług Firebase

Instancja Firebase Functions korzysta z regionu:

us-central1

Metody

isLoggedIn

```
fun isLoggedIn(): Boolean
```

Sprawdza, czy w FirebaseAuth znajduje się aktualnie zalogowany użytkownik.

Zwracana wartość	Opis
true	Użytkownik jest zalogowany
false	Użytkownik nie jest zalogowany

currentUid

```
fun currentUid(): String?
```

Zwraca UID aktualnie zalogowanego użytkownika.

Jeżeli użytkownik nie jest zalogowany, metoda zwraca null.

currentAuthProvider

```
fun currentAuthProvider(): AccountAuthProvider
```

Określa dostawcę użytego do uwierzytelnienia konta.

Możliwe wartości:

Wartość	Znaczenie
EMAIL	Konto korzysta z adresu e-mail i hasła
GOOGLE	Konto korzysta z logowania Google
UNKNOWN	Nie udało się rozpoznać dostawcy

login

```
suspend fun login(
    email: String,
    password: String
): Boolean
```

Loguje użytkownika za pomocą adresu e-mail i hasła.

Adres e-mail jest oczyszczany z białych znaków za pomocą `trim()`.

Wynik	Znaczenie
-------	-----------

true	Logowanie zakończyło się powodzeniem
------	--------------------------------------

false	Wystąpił błąd logowania
-------	-------------------------

Metoda komunikuje się bezpośrednio z Firebase Authentication.

register

```
suspend fun register(  
    email: String,  
    password: String  
): Boolean
```

Tworzy konto za pomocą adresu e-mail i hasła.

Po utworzeniu konta wywołuje:

```
userRepository.createUserIfNotExists(firebaseUser)
```

Wynik	Znaczenie
-------	-----------

true	Konto zostało utworzone
------	-------------------------

false	Wystąpił błąd rejestracji
-------	---------------------------

loginWithGoogle

```
suspend fun loginWithGoogle(context: Context)
```

Pobiera dane logowania Google za pomocą CredentialManager, tworzy dane uwierzytelniające Firebase i loguje użytkownika.

Po poprawnym logowaniu tworzy lub aktualizuje dokument użytkownika w Firestore.

reauthenticateWithPassword

```
suspend fun reauthenticateWithPassword(  
    password: String  
)
```

Ponownie uwierzytelnia użytkownika za pomocą hasła.

Po poprawnej reautoryzacji wymusza wygenerowanie nowego tokenu Firebase ID:

```
currentUser.getIdToken(true)
```

Metoda jest używana przed operacjami wymagającymi świeżego uwierzytelnienia, takimi jak usunięcie konta.

```
reauthenticateWithGoogle  
suspend fun reauthenticateWithGoogle(  
    context: Context  
)
```

Ponownie uwierzytelnia użytkownika za pomocą konta Google i wymusza wygenerowanie nowego tokenu Firebase ID.

```
deleteAccount  
suspend fun deleteAccount()
```

Wywołuje backendową funkcję:

`deleteAccount`

Dane wysyłane do backendu:

```
{  
    "confirm": true  
}
```

Przebieg działania:

18. Sprawdzenie, czy użytkownik jest zalogowany.
19. Wymuszenie odświeżenia tokenu Firebase ID.
20. Wywołanie funkcji `deleteAccount`.
21. Oczekiwanie na zakończenie usuwania konta.
22. Wyczyszczenie lokalnego stanu Firebase Authentication.

Możliwe błędy są przekazywane jako wyjątki.

```
logout  
fun logout()
```

Wylogowuje użytkownika lokalnie za pomocą:

```
auth.signOut()
```

7.2. FriendRepository

Odpowiedzialność

Repozytorium odpowiada za pobieranie użytkowników, listy znajomych i zaproszeń oraz za wywoływanie backendowych funkcji zarządzających relacjami znajomości.

Zależności

Zależność	Zastosowanie
FirebaseAuth	Pobranie UID aktualnego użytkownika
FirebaseFirestore	Odczyt użytkowników, znajomych i zaproszeń
FirebaseFunctions	Wysyłanie, akceptowanie, odrzucanie i usuwanie znajomych

Kolekcje

users
friendRequests
users/{uid}/friends

Metody

getUsersForFriendSearch

suspend **fun** getUsersForFriendSearch(): List<UserProfile>

Pobiera użytkowników, których można wyświetlić w wynikach wyszukiwania znajomych.

Z wyników usuwani są:

- aktualnie zalogowany użytkownik,
- obecni znajomi,
- użytkownicy z oczekującymi zaproszeniami przychodzącymi,
- użytkownicy z oczekującymi zaproszeniami wychodzącymi.

Repozytorium pobiera maksymalnie:

200 użytkowników

Operacje Firestore:

Operacja	Ścieżka
Odczyt	users/{uid}/friends
Odczyt	friendRequests, gdzie toUid == uid i status == pending
Odczyt	friendRequests, gdzie fromUid == uid i status == pending

Operacja	Ścieżka
Odczyt	users, limit 200

getFriends

```
suspend fun getFriends(): List<UserProfile>
```

Pobiera identyfikatory znajomych z podkolekcji aktualnego użytkownika, a następnie pobiera pełny dokument każdego użytkownika.

Operacje Firestore:

Operacja	Ścieżka
Odczyt	users/{uid}/friends
Odczyt	users/{friendUid}

getIncomingRequests

```
suspend fun getIncomingRequests(): List<FriendRequestData>
```

Pobiera oczekujące zaproszenia, których odbiorcą jest aktualny użytkownik.

Zapytanie:

```
toUid == currentUid
status == pending
```

Operacja wykonywana jest bezpośrednio na kolekcji:

friendRequests

getOutgoingRequests

```
suspend fun getOutgoingRequests(): List<FriendRequestData>
```

Pobiera oczekujące zaproszenia wysłane przez aktualnego użytkownika.

Zapytanie:

```
fromUid == currentUid
status == pending
```

sendFriendRequest

```
suspend fun sendFriendRequest(toUid: String)
```

Wywołuje funkcję backendową:

sendFriendRequest

Dane wysyłane do backendu:

```
{  
  "toUid": "recipientUid"  
}
```

acceptFriendRequest

```
suspend fun acceptFriendRequest(  
    requestId: String  
)
```

Wywołuje funkcję backendową:

acceptFriendRequest

Dane wysyłane do backendu:

```
{  
  "requestId": "uidA_uidB"  
}
```

rejectFriendRequest

```
suspend fun rejectFriendRequest(  
    requestId: String  
)
```

Wywołuje funkcję backendową:

rejectFriendRequest

Dane wysyłane do backendu:

```
{  
  "requestId": "uidA_uidB"  
}
```

removeFriend

```
suspend fun removeFriend(  
    friendUid: String  
)
```

Wywołuje funkcję backendową:

removeFriend

Dane wysyłane do backendu:

```
{  
  "friendUid": "friendUid"  
}
```

7.3. GuessRepository

Odpowiedzialność

Repozytorium odpowiada za przesyłanie odpowiedzi użytkownika, pobieranie wcześniejszej odpowiedzi oraz pobieranie parametrów początkowego widoku mapy.

Zależności

Zależność	Zastosowanie
FirebaseFunctions	Wywoływanie funkcji zgadywania
FirebaseFirestore	Pobieranie zapisanego wyniku
FirebaseAuth	Pobranie UID aktualnego użytkownika

Metody

submitGuess

```
suspend fun submitGuess(  
    postId: String,  
    guessedLatitude: Double,  
    guessedLongitude: Double,  
    gameMode: GameModeType = GameModeType.CITY  
): SubmitGuessResult
```

Wywołuje funkcję backendową:

submitGuess

Dane wysyłane do backendu:

```
{  
  "postId": "postId",  
  "guessedLatitude": 50.0189,  
  "guessedLongitude": 21.9835,  
  "gameMode": "CITY"  
}
```

Mapowanie odpowiedzi:

Pole backendu	Typ
guessId	String
postId	String

Pole backendu	Typ
gameMode	String
distanceMeters	Number
points	Number
maxPoints	Number
maxScoringDistanceMeters	Number
scoringVersion	Number
guessedLatitude	Number
guessedLongitude	Number
realLatitude	Number
realLongitude	Number

Odpowiedź backendu jest rzutowana na:

Map<*, *>

Brak wymaganych pól lub ich niepoprawne typy mogą spowodować błąd rzutowania.

getMyGuessForPost

```
suspend fun getMyGuessForPost(  
    postId: String  
): SubmitGuessResult?
```

Pobiera wcześniejszą odpowiedź aktualnego użytkownika dla wskazanego posta.

Identyfikator dokumentu:

{uid}_{postId}

Ścieżka:

guesses/{uid}_{postId}

Jeżeli użytkownik nie jest zalogowany albo dokument nie istnieje, metoda zwraca null.

getGuessMapPreview

```
suspend fun getGuessMapPreview(  
    postId: String,  
    gameMode: GameModeType  
): GuessMapPreview
```

Wywołuje funkcję backendową:

getGuessMapPreview

Dane wysyłane do backendu:

```
{  
  "postId": "postId",  
  "gameMode": "CITY"  
}
```

Mapowane pola odpowiedzi:

Pole backendu	Typ
postId	String
gameMode	String
initialMapCenterLatitude	Number
initialMapCenterLongitude	Number
initialMapDiameterMeters	Number

7.4. UserRepository

Odpowiedzialność

Repozytorium odpowiada za tworzenie i aktualizowanie dokumentów użytkowników, pobieranie profilu oraz zmianę zdjęcia profilowego.

Zależności

Zależność	Zastosowanie
FirebaseFirestore	Operacje na dokumentach użytkowników
FirebaseStorage	Przechowywanie zdjęć profilowych
FirebaseUser	Dane użytkownika z Firebase Authentication

Metody

createUserIfNotExists

```
suspend fun createUserIfNotExists(  
    firebaseUser: FirebaseUser  
)
```

Sprawdza, czy dokument użytkownika istnieje.

Jeżeli dokument nie istnieje, tworzony jest z wartościami początkowymi.

Jeżeli dokument istnieje, aktualizowane są pola:

lastLoginAt
updatedAt

Początkowe statystyki:

```
{  
  "pointsTotal": 0,  
  "guessesCount": 0,  
  "postsCount": 0,  
  "friendsCount": 0,  
  "bestGuessMeters": null  
}
```

Nickname jest ustalany na podstawie pierwszej dostępnej wartości:

- 23. `firebaseUser.displayName`,
- 24. część adresu e-mail przed znakiem @,
- 25. wartość `Player`.

```
getUserProfile  
suspend fun getUserProfile(  
    uid: String  
): UserProfile?
```

Pobiera dokument:

`users/{uid}`

Jeżeli dokument nie istnieje lub nie może zostać zmapowany, metoda zwraca `null`.

```
updateProfilePicture  
suspend fun updateProfilePicture(  
    uid: String,  
    imageUri: Uri  
): String
```

Przesyła nowe zdjęcie profilowe do Firebase Storage.

Ścieżka pliku:

`avatars/{uid}/profile_{timestamp}.jpg`

Po przesłaniu pliku aktualizowane są pola użytkownika:

`photoUrl`
`avatarPath`
`updatedAt`

Metoda zwraca publiczny URL przesłanego zdjęcia.

7.5. PostRepository

Odpowiedzialność

Repozytorium odpowiada za pobieranie postów znajomych, własnych postów, zgadniętych postów, dodawanie postów i usuwanie własnych postów.

Zależności

Zależność	Zastosowanie
FirebaseAuth	Pobranie aktualnego użytkownika
FirebaseFirestore	Operacje na postach, użytkownikach, znajomych i odpowiedziach
FirebaseStorage	Przechowywanie i usuwanie zdjęć postów
PostFactory	Mapowanie dokumentów Firestore i tworzenie danych posta

Metody

getPosts

```
suspend fun getPosts(): List<Post>
```

Pobiera aktywne dane postów znajomych aktualnego użytkownika.

Przebieg działania:

26. Pobranie UID znajomych.
27. Pobranie identyfikatorów postów już zgadniętych.
28. Pobranie postów znajomych.
29. Usunięcie postów wcześniej zgadniętych.
30. Posortowanie postów malejąco według createdAt.
31. Zamiana dokumentów Firestore na dane aplikacji.
32. Dołączenie aktualnych danych autora.

Ze względu na ograniczenia zapytania whereIn, UID znajomych są dzielone na grupy po:

30 elementów

Operacje Firestore:

Operacja	Ścieżka
Odczyt	users/{uid}/friends
Odczyt	guesses, gdzie userId == uid
Odczyt	posts, gdzie userId in friendUids
Odczyt	users, gdzie identyfikator dokumentu znajduje się w liście autorów

getMyPosts

```
suspend fun getMyPosts(): List<Post>
```

Pobiera posty aktualnego użytkownika.

Zapytanie:

```
userId == currentUid  
orderBy createdAt DESC
```

Operacja może wymagać indeksu złożonego Firestore dla pól:

```
userId  
createdAt
```

getMyGuessedPosts

```
suspend fun getMyGuessedPosts(): List<GuessedPost>
```

Pobiera odpowiedzi aktualnego użytkownika, następnie pobiera odpowiadające im posty.

Przebieg działania:

33. Pobranie dokumentów z guesses.
34. Posortowanie ich malejąco według createdAt.
35. Pobranie postów wskazanych przez postId.
36. Dołączenie aktualnych danych autorów.
37. Połączenie posta z liczbą punktów i odległością.

Jeżeli powiązany post nie istnieje, wpis jest pomijany.

addPost

```
suspend fun addPost(  
    bitmap: Bitmap,  
    latitude: Double,  
    longitude: Double  
): Post
```

Dodaje nowy post aktualnego użytkownika.

Przebieg działania:

38. Sprawdzenie, czy użytkownik jest zalogowany.
39. Pobranie danych użytkownika.
40. Wygenerowanie identyfikatora dokumentu posta.

41. Konwersja obrazu do JPEG.
42. Przesłanie obrazu do Firebase Storage.
43. Pobranie URL obrazu.
44. Utworzenie dokumentu posta.
45. Zwiększenie licznika `stats.postsCount`.
46. Zwrócenie danych utworzonego posta.

Obraz jest kompresowany do formatu:

JPEG, jakość 85%

Ścieżka pliku:

`posts/{userId}/{postId}.jpg`

Operacje Firestore:

Operacja	Ścieżka	Opis
Odczyt	<code>users/{userId}</code>	Pobranie nazwy i zdjęcia autora
Zapis	<code>posts/{postId}</code>	Utworzenie posta
Aktualizacja	<code>users/{userId}</code>	Zwiększenie <code>stats.postsCount</code>

deleteMyPost

```
suspend fun deleteMyPost(  
    postId: String  
)
```

Usuwa post należący do aktualnego użytkownika.

Operacja Firestore wykonywana jest w transakcji.

Walidacja:

47. Użytkownik musi być zalogowany.
48. Dokument posta musi istnieć.
49. Pole `userId` musi być zgodne z UID aktualnego użytkownika.

Operacje Firestore:

Operacja	Ścieżka
Odczyt	<code>posts/{postId}</code>
Usunięcie	<code>posts/{postId}</code>
Aktualizacja	<code>users/{uid}</code>

Aktualizowane pola użytkownika:

```
stats.postsCount = increment(-1)
updatedAt = serverTimestamp
```

Po zakończeniu transakcji usuwany jest plik:

```
posts/{userId}/{postId}.jpg
```

Brak pliku w Storage nie powoduje przerwania operacji.

Usunięcie posta nie usuwa w tej metodzie dokumentów z kolekcji `guesses`, które odnoszą się do tego posta.

7.6. SettingsRepository

Odpowiedzialność

Repozytorium odpowiada za lokalne ustawienia aplikacji przechowywane za pomocą Android DataStore.

Repozytorium nie komunikuje się z Firebase ani z funkcjami backendowymi.

Zależności

Zależność	Zastosowanie
Context	Dostęp do DataStore
PreferencesKeys	Klucze ustawień
DataStore	Lokalny zapis preferencji

Odczytywane ustawienia

Właściwość	Typ	Wartość domyślna
darkModeFlow	Flow<Boolean>	false
useMilesFlow	Flow<Boolean>	false
isLoggedInFlow	Flow<Boolean>	false

W przypadku błędu `IOException` emitowany jest pusty zestaw preferencji.

Inne wyjątki są przekazywane dalej.

Metody zapisu

```
setDarkMode
suspend fun setDarkMode(
    enabled: Boolean
)
```

Zapisuje ustawienie trybu ciemnego.

```
setUseMiles
suspend fun setUseMiles(
    enabled: Boolean
)
```

Zapisuje ustawienie jednostki odległości.

```
setLoggedIn
suspend fun setLoggedIn(
    isLoggedIn: Boolean
)
```

Zapisuje lokalną informację o stanie zalogowania.

8. Struktura danych Cloud Firestore

8.1. Kolekcja `users`

Ścieżka

`users/{uid}`

Identyfikator dokumentu jest równy UID użytkownika z Firebase Authentication.

Pola dokumentu

Pole	Typ Firestore	Wymagane	Opis
<code>uid</code>	<code>string</code>	Tak	UID użytkownika
<code>email</code>	<code>string</code>	Tak	Adres e-mail
<code>displayName</code>	<code>string</code>	Tak	Nazwa wyświetlana
<code>nickname</code>	<code>string</code>	Tak	Nick użytkownika
<code>photoUrl</code>	<code>string</code> lub <code>null</code>	Tak	Publiczny URL zdjęcia profilowego
<code>avatarPath</code>	<code>string</code> lub <code>null</code>	Tak	Ścieżka pliku w Firebase Storage
<code>isBanned</code>	<code>boolean</code>	Tak	Informacja o blokadzie konta
<code>stats</code>	<code>map</code>	Tak	Statystyki użytkownika

Pole	Typ Firestore	Wymagane	Opis
createdAt	timestamp	Tak	Data utworzenia konta
updatedAt	timestamp	Tak	Data ostatniej aktualizacji
lastLoginAt	timestamp	Tak	Data ostatniego logowania

Mapa stats

Pole	Typ Firestore	Opis
pointsTotal	int64	Łączna liczba punktów
guessesCount	int64	Liczba wykonanych zgadywań
postsCount	int64	Liczba dodanych postów
friendsCount	int64	Liczba znajomych
bestGuessMeters	double lub null	Najmniejsza uzyskana odległość

Przykład

```
{
  "uid": "NlxMZQG7pRhCTucv51phWPdy37J3",
  "email": "ginger@gmail.com",
  "displayName": "",
  "nickname": "ginger",
  "photoUrl": null,
  "avatarPath": null,
  "isBanned": false,
  "stats": {
    "pointsTotal": 0,
    "guessesCount": 0,
    "postsCount": 0,
    "friendsCount": 0,
    "bestGuessMeters": null
  },
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp",
  "lastLoginAt": "serverTimestamp"
}
```

Repozytoria korzystające z kolekcji

Repozytorium	Operacje
UserRepository	Odczyt, zapis, aktualizacja
FriendRepository	Odczyt

Repozytorium	Operacje
PostRepository	Odczyt, aktualizacja
AuthRepository	Pośrednio przez UserRepository
GuessRepository	Brak bezpośrednich operacji

8.2. Podkolekcja `friends`

Ścieżka

`users/{uid}/friends/{friendUid}`

Identyfikator dokumentu jest równy UID znajomego.

Relacja znajomości jest zapisywana dwukierunkowo:

```
users/{uidA}/friends/{uidB}
users/{uidB}/friends/{uidA}
```

Pola dokumentu

Pole	Typ Firestore	Wymagane	Opis
<code>uid</code>	<code>string</code>	Tak	UID znajomego
<code>nickname</code>	<code>string</code>	Tak	Nick znajomego
<code>displayName</code>	<code>string</code>	Tak	Nazwa wyświetlana znajomego
<code>photoUrl</code>	<code>string</code> lub <code>null</code>	Tak	URL zdjęcia znajomego
<code>addedAt</code>	<code>timestamp</code>	Tak	Data utworzenia relacji

Przykład

```
{
  "uid": "XkKhzx6XNJZJSIIeFdXB4EAP1Lh1",
  "nickname": "rendor535",
  "displayName": "rendor535",
  "photoUrl": "https://firebasestorage.googleapis.com/...",
  "addedAt": "serverTimestamp"
}
```

8.3. Kolekcja `friendRequests`

Ścieżka

`friendRequests/{requestId}`

Identyfikator dokumentu jest generowany przez posortowanie UID obu użytkowników:

`{mniejszeUid}_{większeUid}`

Dla jednej pary użytkowników istnieje jeden dokument zaproszenia.

Pola dokumentu

Pole	Typ Firestore	Wymagane	Opis
fromUid	string	Tak	UID nadawcy
toUid	string	Tak	UID odbiorcy
fromNickname	string	Tak	Nick nadawcy
fromDisplayName	string	Tak	Nazwa nadawcy
fromPhotoUrl	string lub null	Tak	Zdjęcie nadawcy
toNickname	string	Tak	Nick odbiorcy
toDisplayName	string	Tak	Nazwa odbiorcy
toPhotoUrl	string lub null	Tak	Zdjęcie odbiorcy
status	string	Tak	Status zaproszenia
createdAt	timestamp	Tak	Data utworzenia
updatedAt	timestamp	Tak	Data ostatniej zmiany

Możliwe statusy

Status	Znaczenie
pending	Zaproszenie oczekuje na decyzję
accepted	Zaproszenie zostało zaakceptowane
rejected	Zaproszenie zostało odrzucone
removed	Relacja znajomości została usunięta

Przykład

```
{
  "fromUid": "NlxMZQG7pRhCTucv51phWPdy37J3",
  "toUid": "lLD9uSB0i5SS42NBZ0MJMQ56mDA3",
  "fromNickname": "ginger",
  "fromDisplayName": "ginger",
  "fromPhotoUrl": null,
  "toNickname": "Marcin Wiśniowski",
  "toDisplayName": "Marcin Wiśniowski",
  "toPhotoUrl": "https://firebasestorage.googleapis.com/...",
  "status": "removed",
  "createdAt": "serverTimestamp",
  "updatedAt": "serverTimestamp"
}
```

Funkcje korzystające z kolekcji

Funkcja	Operacja
sendFriendRequest	Odczyt i zapis

Funkcja	Operacja
acceptFriendRequest	Odczyt i aktualizacja
rejectFriendRequest	Odczyt i aktualizacja
removeFriend	Zapis z opcją merge
deleteAccount	Odczyt i usunięcie

8.4. Kolekcja posts

Ścieżka

posts/{postId}

Identyfikator dokumentu jest generowany automatycznie przez Firestore.

Pola dokumentu

Pole	Typ Firestore	Wymagane	Opis
userId	string	Tak	UID autora
user	string	Tak	Nazwa autora zapisana podczas dodawania posta
imageUrl	string	Tak	URL zdjęcia w Firebase Storage
latitude	double	Tak	Szerokość geograficzna
longitude	double	Tak	Długość geograficzna
createdAt	timestamp	Tak	Data utworzenia posta
isActive	boolean	Tak	Informacja o aktywności posta
authorPhotoUrl	string lub null	Nie	Zdjęcie autora zapisane w poście
gameMode	string	Nie	Tryb gry; brak pola oznacza CITY

Przykład

```
{
  "userId": "eoqlkWkkDfWQvjYoOGIksUSVso23",
  "user": "Sławomir Zdunek",
  "imageUrl": "https://firebasestorage.googleapis.com/...",
  "latitude": 50.0201652,
```

```
"longitude": 21.9841071,  
"createdAt": "serverTimestamp",  
"isActive": true  
}
```

Funkcje i repozytoria korzystające z kolekcji

Element	Operacja
submitGuess	Odczyt
getGuessMapPreview	Odczyt
deleteAccount	Odczyt i usunięcie
PostRepository	Odczyt, zapis i usunięcie

8.5. Kolekcja guesses

Ścieżka

guesses/{uid}_{postId}

Identyfikator dokumentu jest złożeniem UID użytkownika i identyfikatora posta.

Zapewnia to jeden dokument odpowiedzi użytkownika dla jednego posta.

Pola dokumentu

Pole	Typ Firestore	Wymagane	Opis
id	string	Tak	Identyfikator dokumentu
userId	string	Tak	UID użytkownika
postId	string	Tak	Identyfikator posta
gameMode	string	Tak	Tryb gry
guessedLatitude	double	Tak	Wybrana szerokość geograficzna
guessedLongitude	double	Tak	Wybrana długość geograficzna
realLatitude	double	Tak	Rzeczywista szerokość geograficzna
realLongitude	double	Tak	Rzeczywista długość geograficzna
distanceMeters	double	Tak	Odległość od

Pole	Typ Firestore	Wymagane	Opis
			właściwej lokalizacji
points	int64	Tak	Liczba zdobytych punktów
maxPoints	int64	Tak	Maksymalna liczba punktów
maxScoringDistanceMeters	int64 lub double	Tak	Maksymalna punktowana odległość
scoringVersion	int64	Tak	Wersja systemu punktacji
createdAt	timestamp	Tak	Data wykonania odpowiedzi

Przykład

```
{
  "id": "P9SvNZiSb9fajbTToRlU0zp8Sjx2_MaVhi07NbviGQtqXer6e",
  "userId": "P9SvNZiSb9fajbTToRlU0zp8Sjx2",
  "postId": "MaVhi07NbviGQtqXer6e",
  "gameMode": "CITY",
  "guessedLatitude": 50.01898434032029,
  "guessedLongitude": 21.98354315041874,
  "realLatitude": 50.0199646,
  "realLongitude": 21.9832812,
  "distanceMeters": 110.59493388195747,
  "points": 9983,
  "maxPoints": 10000,
  "maxScoringDistanceMeters": 2000,
  "scoringVersion": 1,
  "createdAt": "serverTimestamp"
}
```

Funkcje i repozytoria korzystające z kolekcji

Element	Operacja
submitGuess	Odczyt i zapis
deleteAccount	Odczyt i usunięcie
GuessRepository	Odczyt
PostRepository	Odczyt

9. Struktura Firebase Storage

9.1. Zdjęcia profilowe

Ścieżka

avatars/{uid}/profile_{timestamp}.jpg

Operacje

Element	Operacja
UserRepository.updateProfilePicture	Zapis
deleteAccount	Usunięcie wszystkich plików użytkownika

9.2. Zdjęcia postów

Ścieżka

posts/{uid}/{postId}.jpg

Operacje

Element	Operacja
PostRepository.addPost	Zapis
PostRepository.deleteMyPost	Usunięcie pojedynczego pliku
deleteAccount	Usunięcie wszystkich plików użytkownika

10. Powiązania funkcji backendowych z repozytoriami

Funkcja backendowa	Repozytorium	Metoda repozytorium	Cel
sendFriendRequest	FriendRepository	sendFriendRequest(toUid)	Wysłanie zaproszenia
acceptFriendRequest	FriendRepository	acceptFriendRequest(requestId)	Akceptacja zaproszenia
rejectFriendRequest	FriendRepository	rejectFriendRequest(requestId)	Odrzucenie zaproszenia
removeFriend	FriendRepository	removeFriend(friendUid)	Usunięcie relacji znajomości
submitGuess	GuessRepository	submitGuess(...)	Zapis odpowiedzi i obliczenie wyniku
getGuessMapPreview	GuessRepository	getGuessMapPreview	Pobranie

Funkcja backendowa	Repozytorium	Metoda repozytorium (...)	Cel
			początkowych parametrów mapy
deleteAccount	AuthRepository	deleteAccount()	Trwałe usunięcie konta
health	Brak	Brak	Kontrola dostępności backendu

11. Operacje wykonywane bezpośrednio na Firebase

Repozytorium	Usługa	Operacje
AuthRepository	Firebase Authentication	Logowanie, rejestracja, reautoryzacja, wylogowanie
FriendRepository	Cloud Firestore	Pobieranie użytkowników, znajomych i zaproszeń
GuessRepository	Cloud Firestore	Pobieranie zapisanej odpowiedzi
UserRepository	Cloud Firestore	Tworzenie, pobieranie i aktualizacja użytkownika
UserRepository	Firebase Storage	Przesyłanie zdjęcia profilowego
PostRepository	Cloud Firestore	Pobieranie, tworzenie i usuwanie postów
PostRepository	Firebase Storage	Przesyłanie i usuwanie zdjęć postów
SettingsRepository	Android DataStore	Lokalny zapis ustawień

12. Podsumowanie kontraktów

Funkcja	Repozytorium	Uwierzytelnienie	Dane wejściowe	Odpowiedź
health	Brak	Nie	Brak	Stan backendu
sendFriendRequest	FriendRepository	Tak	toUid	success, requestId
acceptFriendRequest	FriendRepository	Tak	requestId	success

Funkcja	Repozytorium	Uwierzytelnienie	Dane wejściowe	Odpowiedź
rejectFriendRequest	FriendRepository	Tak	requestId	success
removeFriend	FriendRepository	Tak	friendUid	success
getGuessMapPreview	GuessRepository	Tak	postId, gameMode	Parametry mapy
submitGuess	GuessRepository	Tak	postId, współrzędne, gameMode	Wynik odpowiedzi
deleteAccount	AuthRepository	Tak	confirm	success